Knowledge-Graph-Guided Fine-Tuning of Embedding Models for Mathematical Document Retrieval

Abstract

We present a method for improving semantic search over mathematical research papers by fine-tuning embedding models using contrastive learning, guided by a knowledge graph extracted from the corpus. General-purpose embedding models (e.g., OpenAI's `text-embedding-3-small`) and even scientific embedding models (SPECTER2, SciNCL) perform poorly on mathematical retrieval tasks because they lack understanding of the semantic relationships between mathematical concepts. Our approach exploits an existing knowledge graph — whose nodes are mathematical concepts and whose edges encode relationships such as *generalizes*, *proves*, and *is_instance_of* — to automatically generate training data for contrastive fine-tuning. We benchmark baseline models against our fine-tuned model on a retrieval task over 4,794 paper chunks spanning 75 papers in algebraic combinatorics, and demonstrate that domain-specific fine-tuning significantly outperforms all baselines. The method is general: given any corpus of mathematical papers and a knowledge graph over their concepts, the same pipeline produces a domain-adapted embedding model.

1 Introduction

The increasing volume of mathematical literature makes automated retrieval tools indispensable for researchers. A common approach is *retrieval-augmented generation* (RAG): chunk papers into passages, embed them in a vector space, and retrieve relevant passages via nearest-neighbor search over embeddings. The quality of retrieval depends critically on the embedding model's ability to capture *mathematical semantic similarity* — the idea that a query like "Rogers–Ramanujan identities" should retrieve not only passages containing that exact phrase but also passages discussing Bailey's lemma, q-series transformations, and partition identities.

General-purpose embedding models are trained on broad web text and lack this kind of domain knowledge. Scientific embedding models such as SPECTER2 [2] and SciNCL [3] are trained on citation graphs from Semantic Scholar, but mathematics is underrepresented in their training data, and they are optimized for *paper-to-paper* similarity rather than *concept-to-passage* retrieval.

We address this gap by fine-tuning an embedding model specifically for mathematical concept retrieval. Our key insight is that a **knowledge graph** (KG) extracted from the corpus provides exactly the supervision signal needed for contrastive learning:

- Each KG concept (e.g., "Macdonald polynomials") maps to specific papers, and hence to specific text chunks. These form *positive pairs* for contrastive training.
- KG edges (e.g., "Bailey's lemma *generalizes* Rogers–Ramanujan identities") provide *cross-concept positives* that teach the model about mathematical relationships.
- In-batch negatives from unrelated concepts provide the contrastive signal automatically.

This paper makes the following contributions:

1. A benchmark comparing general-purpose and scientific embedding models on mathematical concept retrieval (Section 4).
2. A method for automatically generating contrastive training data from a knowledge graph (Section 5).

3. A fine-tuned embedding model that outperforms all baselines on our benchmark (Section 6).
4. An open-source pipeline[1] that can be applied to any mathematical corpus with an associated knowledge graph.

2 Related Work

Scientific document embeddings. SPECTER [1] introduced citation-based contrastive learning for scientific document embeddings, training on (paper, cited paper, non-cited paper) triplets. SPECTER2 [2] extended this to 6 million citation triplets across 23 fields of study and introduced task-specific adapters (proximity, classification, regression). SciNCL [3] improved on SPECTER by using citation graph *neighborhood* sampling for harder negatives. All three models use SciBERT [4] as their backbone and produce 768-dimensional embeddings.

Mathematics-specific models. MathBERT [5] pre-trained BERT on mathematical curricula and arXiv abstracts, but only with masked language modeling — it was not contrastively trained for retrieval. No widely adopted embedding model exists that is specifically trained for mathematical semantic similarity.

Contrastive fine-tuning. The sentence-transformers framework [6] provides `MultipleNegativesRankingLos` (MNRL), which treats all other examples in a batch as negatives. Matryoshka Representation Learning [7] trains embeddings so that any prefix of the full vector is itself a useful embedding, enabling flexible dimensionality–quality tradeoffs at inference.

3 Data

3.1 Corpus

Our corpus consists of 75 research papers in algebraic combinatorics, q-series, and related areas, sourced from arXiv. Papers are chunked into passages of up to 1,500 characters with 200-character overlap, yielding **4,794 chunks**. The chunks are stored in a ChromaDB vector database with embeddings from OpenAI's `text-embedding-3-small` (1536-dim).

3.2 Knowledge graph

A knowledge graph was constructed by having GPT-4o-mini extract concepts and relationships from representative chunks (first two and last two) of each paper [8]. After normalization and deduplication, the KG contains:

- **559 concepts** (218 objects, 92 theorems, 77 definitions, 56 techniques, 28 persons, 26 formulas, 25 identities, 11 conjectures, and others)
- **486 edges** with typed relationships (*related_to*: 110, *uses*: 78, *generalizes*: 54, *is_instance_of*: 45, *implies*: 40, *defines*: 39, and others)
- Coverage of all 75 papers

4 Benchmark

4.1 Ground truth construction

We construct a retrieval benchmark from the KG. For each concept c with at least min_degree = 2 matched papers in the corpus:

[1] Available at https://github.com/XXX/embeddings

- **Query**: the concept's display name (e.g., "Rogers–Ramanujan identities")
- **Relevant documents**: all chunks from the concept's source papers

This yields **108 queries**. The ground truth is approximate — not every chunk in a relevant paper directly discusses the concept — but this bias is consistent across models, making relative comparisons valid.

4.2 Metrics

We report:

- **MRR** (Mean Reciprocal Rank): the average inverse rank of the first relevant result.
- **NDCG@k** (Normalized Discounted Cumulative Gain): measures ranking quality with position-dependent discounting.
- **Recall@k**: fraction of relevant documents retrieved in the top k. Note that Recall@k appears low because relevant sets are large (often 100+ chunks per concept); MRR and NDCG are the meaningful comparison metrics.

All metrics are computed using a Rust implementation with rayon parallelism for batch kNN and metric aggregation [9].

4.3 Baseline results

Table 1: Baseline embedding model comparison on mathematical concept retrieval. All models evaluated on 108 queries over 4,794 chunks.

Model	Dim	R@5	R@10	R@20	MRR	NDCG@10
`openai-small`	1536	0.010	0.019	0.037	**0.461**	**0.324**
SPECTER2 (proximity)	768	0.007	0.013	0.024	0.360	0.225
SciNCL	768	0.006	0.012	0.024	0.306	0.205
Math-Embed (ours)	768	—	—	—	—	—

The general-purpose OpenAI model outperforms both scientific models by a wide margin (28% higher MRR than SPECTER2, 51% higher than SciNCL). This is notable because SPECTER2 was trained on 6 million scientific citation triplets — yet it underperforms a model with no scientific specialization. We attribute this to two factors:

1. **Dimensionality**: OpenAI's 1536-dim space has more capacity than the 768-dim BERT-based models.
2. **Task mismatch**: SPECTER2 and SciNCL were trained for paper-to-paper similarity (title + abstract), not concept-to-chunk retrieval. A query like "Rogers–Ramanujan identities" is not a paper title — it is a mathematical concept name, and retrieving relevant passages requires understanding what that concept means.

5 Training Data from Knowledge Graphs

We generate contrastive training data automatically from the KG and corpus.

5.1 Direct pairs

For each concept c with papers $P_1, \ldots, P_m$ in the KG, and each paper P_j with chunks $\{d_{j,1}, \ldots, d_{j,n_j}\}$ in the corpus:

$$\text{Pairs}_{\text{name}}(c) = \{(\texttt{name}(c),\ d_{j,k}) : j \in [m],\ k \in [n_j]\} \tag{1}$$

$$\text{Pairs}_{\text{desc}}(c) = \{(\texttt{desc}(c),\ d_{j,k}) : j \in [m],\ k \in [n_j]\} \tag{2}$$

Using both the concept name and its description as anchors provides anchor diversity: short anchors (e.g., "Macdonald polynomials") train exact-match retrieval, while longer descriptions (e.g., "A family of orthogonal symmetric polynomials generalizing Schur functions") train paraphrase retrieval.

We cap at 20 chunks per concept to prevent over-representation of high-degree concepts.

5.2 Edge pairs

For each edge (c_1, c_2, r) in the KG with relation r (e.g., *generalizes*, *uses*):

$$\text{Pairs}_{\text{edge}}(c_1, c_2) = \{(\texttt{name}(c_1),\ d) : d \in \text{chunks}(c_2)\} \cup \{(\texttt{name}(c_2),\ d) : d \in \text{chunks}(c_1)\} \quad (3)$$

These cross-concept pairs teach the model that mathematically related concepts should embed nearby. For example, if "Bailey's lemma" *generalizes* "Rogers–Ramanujan identities," then chunks about Rogers–Ramanujan should be somewhat relevant to queries about Bailey's lemma.

We cap at 5 chunks per edge direction to prevent edge pairs from dominating the dataset.

5.3 Dataset statistics

Table 2: Training dataset statistics.

Direct pairs (concept → chunk)	21,544
Edge pairs (cross-concept)	4,855
Total unique pairs	25,121
Training set (90%)	22,609
Validation set (10%)	2,512
Unique anchors	1,114

6 Fine-Tuning

6.1 Method

We fine-tune the SPECTER2 base model (`allenai/specter2_base`, 768-dim, SciBERT backbone) using the sentence-transformers framework [6]. Despite SPECTER2's poor off-the-shelf performance on our benchmark, its pre-training on 6 million scientific citation triplets provides a strong initialization for mathematical text — the model already understands scientific language structure, and we teach it mathematical concept semantics on top.

Loss function. We use `MultipleNegativesRankingLoss` (MNRL) wrapped in `MatryoshkaLoss`. MNRL treats all other examples in a batch as negatives, providing $B(B-1)$ negative comparisons per batch of size B without explicit negative mining. MatryoshkaLoss computes the same contrastive loss at multiple embedding truncation points (768, 512, 256, 128 dimensions), training the model to frontload important information into the first dimensions.

Training details.

- Micro-batch size: 8, with gradient accumulation over 4 steps (effective batch size 32, yielding 56 in-batch negative comparisons per micro-batch)
- Max sequence length: 256 tokens (truncating longer chunks)
- Learning rate: 2×10^{-5} with 10% linear warmup
- Epochs: 3 (2,118 optimization steps)
- Duplicate-free batch sampling to maximize negative diversity

- Final model selected after epoch 3 (training loss converged from ∼11 to ∼5)
- Hardware: Apple M-series GPU (MPS backend), ∼4 hours wall time

6.2 Results

Table 3: Final comparison including fine-tuned model. All models evaluated on 108 queries over 4,794 chunks. Best results in bold.

Model	Dim	R@5	R@10	R@20	MRR	NDCG@10
`openai-small`	1536	0.010	0.019	0.037	0.461	0.324
SPECTER2 (proximity)	768	0.007	0.013	0.024	0.360	0.225
SciNCL	768	0.006	0.012	0.024	0.306	0.205
Math-Embed (ours)	768	**0.030**	**0.058**	**0.111**	**0.816**	**0.736**

Our fine-tuned model outperforms all baselines by a wide margin. MRR improves from 0.461 (OpenAI) to **0.816** — a 77% relative improvement, meaning the first relevant result now appears on average at rank ∼1.2 rather than rank ∼2.2. NDCG@10 more than doubles from 0.324 to 0.736, and Recall@20 triples from 0.037 to 0.111.

Remarkably, the fine-tuned model uses half the embedding dimensions (768 vs. 1536) of the OpenAI model yet dramatically outperforms it. The same base model (SPECTER2) that scored worst among baselines (MRR 0.360) becomes the best performer after fine-tuning — a 127% improvement from the same architecture with no additional parameters, demonstrating that the knowledge-graph-derived training signal is highly effective.

7 Discussion

7.1 Why general-purpose models fail at math

The poor performance of SPECTER2 and SciNCL — models explicitly trained on scientific literature — highlights that *scientific* training is not the same as *mathematical* training. These models learn paper-level similarity from citation patterns: "paper A cites paper B, so they should embed nearby." But mathematical retrieval requires a different kind of similarity: understanding that the text "$\sum_{n=0}^{\infty} \frac{q^{n^2}}{(q;q)_n}$" is about the Rogers–Ramanujan identities, even though it contains no occurrence of that phrase.

Standard tokenizers (BERT WordPiece) fragment mathematical notation into meaningless subwords. Fine-tuning cannot fix the tokenizer, but it can teach the model that certain patterns of subword tokens, when they appear together, carry specific mathematical meaning.

7.2 Knowledge graphs as supervision

Our approach requires a knowledge graph, which itself requires an LLM extraction step (GPT-4o-mini in our case). This may seem circular — we use an LLM to generate training data for a different model. But the key insight is that these are *complementary capabilities*:

- The LLM excels at *reading individual passages* and extracting structured information (concepts, relationships), but is too slow and expensive for real-time retrieval over thousands of chunks.
- The embedding model excels at *fast similarity search* over large corpora, but needs training data to learn domain-specific semantics.

The KG is a one-time cost that distills the LLM's understanding into a reusable supervision signal.

7.3 Generalizability

The pipeline is not specific to algebraic combinatorics. Given:

1. A corpus of mathematical papers (any subfield)
2. A knowledge graph over their concepts (extractable by LLM)

the same code produces a domain-adapted embedding model. The fine-tuned model should generalize to new papers in the same mathematical area, since it learns *concept semantics* rather than memorizing specific passages.

8 Conclusion

We demonstrated that general-purpose and scientific embedding models perform poorly on mathematical concept retrieval, and presented a pipeline that automatically generates contrastive training data from a knowledge graph to fine-tune a domain-specific embedding model. Our approach requires no manual annotation — the knowledge graph provides the supervision signal — and produces a portable model that can be deployed in any RAG system for mathematical literature.

Future work includes: (1) scaling to larger mathematical corpora spanning multiple subfields, (2) incorporating mathematical notation awareness into the tokenizer, and (3) exploring whether the fine-tuned model's understanding of mathematical relationships transfers across subfields.

References

[1] A. Cohan, S. Feldman, I. Beltagy, D. Downey, and D. S. Weld, "SPECTER: Document-level representation learning using citation-informed transformers," in *Proc. ACL*, 2020.

[2] A. Singh, M. D'Arcy, A. Cohan, D. Downey, and S. Feldman, "SciRepEval: A multi-format benchmark for scientific document representations," in *Proc. EMNLP*, 2023.

[3] M. Ostendorff, N. Rethmeier, I. Augenstein, B. Gipp, and G. Rehm, "Neighborhood contrastive learning for scientific document representations with citation embeddings," in *Proc. EMNLP*, 2022.

[4] I. Beltagy, K. Lo, and A. Cohan, "SciBERT: A pretrained language model for scientific text," in *Proc. EMNLP*, 2019.

[5] S. Peng, K. Yuan, L. Gao, and Z. Tang, "MathBERT: A pre-trained model for mathematical formula understanding," *arXiv:2105.00377*, 2021.

[6] N. Reimers and I. Gurevych, "Sentence-BERT: Sentence embeddings using Siamese BERT-networks," in *Proc. EMNLP*, 2019.

[7] A. Kusupati, G. Bhatt, A. Rege, M. Wallingford, A. Sinha, V. Ramanujan, W. Howard-Snyder, K. Chen, S. Kakade, P. Jain, and A. Farhadi, "Matryoshka representation learning," in *Proc. NeurIPS*, 2022.

[8] Knowledge graph extraction via LLM-based concept and relationship identification from scientific text, internal methodology.

[9] Custom Rust implementation of batch kNN and IR metrics (Recall@k, MRR, NDCG@k) with rayon parallelism and PyO3 Python bindings.